PYTHON DA END ASSIGNMENT - PYTHON

MARKETING CAMPAIGN PERFORMANCE INSIGHTS

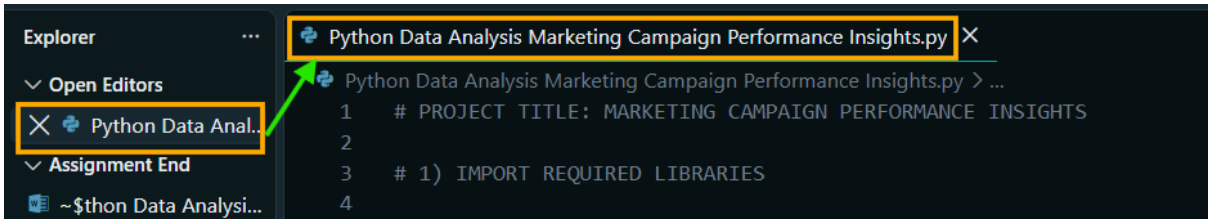
TOOLS: Python / VS Code

PROJECT TITLE: Python DA - Marketing Campaign Performance Insights

PROJECT OBJECTIVE: To clean, analyze, and visualize multi-channel marketing campaign data using Python, Matplotlib, and Seaborn to identify key drivers of Campaign ROI, evaluate channel effectiveness, analyze customer segment performance, and optimize acquisition cost and conversion rates across different geographic locations and demographics.

DATE OF SUBMISSION: 11/11/2026 – 8:45 PM

PROGRAMMING FILE NAME - "Python Data Analysis Marketing Campaign Performance Insights.py"



1) IMPORT REQUIRED LIBRARIES

```
# 1) IMPORT REQUIRED LIBRARIES

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")
```

2) LOAD THE DATASET

2.1) LOAD THE DATASET FROM THE PROVIDED URL

```
# 2.1) Load the dataset from the provided URL
url =
"https://raw.githubusercontent.com/ArchanaInsights/Datasets/main/marketing_cam
paign.csv"

df = pd.read_csv(url)

print("\n========== DATASET LOADED SUCCESSFULLY ==========\n")
print(df.head())
```

3) BASIC DATA EXPLORATION

```
# 3) BASIC DATA EXPLORATION

print("\n========== FIRST 5 ROWS ==========\n")
print(df.head())

print("\n========== NUMBER OF ROWS AND COLUMNS ==========\n")
print(df.shape)

print("\n========== COLUMN NAMES ==========\n")
print(df.columns)

print("\n========== DATA TYPES ==========\n")
print(df.dtypes)

print("\n========== DATASET INFORMATION ==========\n")
df.info()

print("\n========== DESCRIPTIVE STATISTICS ==========\n")
print(df.describe())
```

4) UNIQUE VALUES AND CATEGORY COUNTS

4.1) CHECK FOR UNIQUE VALUES IN KEY COLUMNS

```
# 4.1) Check for unique values in key columns
print("\n========== UNIQUE CAMPAIGN IDs ==========\n")
print(df['Campaign_ID'].nunique())

print("\n========== UNIQUE LOCATIONS ==========\n")
print(df['Location'].unique())

print("\n========== UNIQUE CUSTOMER SEGMENTS ==========\n")
print(df['Customer_Segment'].unique())

print("\n========== CAMPAIGN TYPE COUNTS ==========\n")
print(df['Campaign_Type'].value_counts())
```

4.2) HANDLE CHANNEL_USED / CHANNELS_USED COLUMN NAME DIFFERENCE

```
# 4.2) Handle Channel_Used / Channels_Used column name difference
if 'Channel_Used' in df.columns:
    channel_column = 'Channel_Used'
elif 'Channels_Used' in df.columns:
    channel_column = 'Channels_Used'
else:
```

```
    raise KeyError("Channel_Used or Channels_Used column not found.")

print("\n========== CHANNEL COUNTS ==========\n")
print(df[channel_column].value_counts())
```

5) DATA PREPARATION

5.1) CREATE A COPY OF THE ORIGINAL DATASET

```
# 5.1) Create a copy of the original dataset
df_clean = df.copy()
```

5.2) CONVERT DATE COLUMN INTO DATETIME FORMAT

```
# 5.2) Convert Date column into datetime format
df_clean['Date'] = pd.to_datetime(
    df_clean['Date'],
    format='%d-%m-%Y',  # Explicit format matching '01-01-2023'
    errors='coerce'
)
```

5.3) CLEAN AND CONVERT DURATION (EXTRACT NUMBERS FROM TEXT STRINGS)

```
# 5.3) Clean and convert Duration (extract numbers from text strings)
df_clean['Duration'] =
df_clean['Duration'].astype(str).str.extract(r'(\d+)')[0]
```

5.4) CONVERT ALL REQUIRED NUMERICAL COLUMNS INTO NUMERIC FORMAT

```
# 5.4)Convert all required numerical columns into numeric format
numeric_columns = [
    'Duration',
    'Conversion_Rate',
    'Acquisition_Cost',
    'ROI',
    'Clicks',
    'Impressions',
    'Engagement_Score'
]

for col in numeric_columns:
    df_clean[col] = pd.to_numeric(
        df_clean[col],
        errors='coerce'
    )
```

5.5) REMOVE ROWS WHERE DATE IS MISSING

```
# 5.5) Remove rows where Date is missing
df_clean = df_clean.dropna(subset=['Date'])
```

5.6) FILL MISSING NUMERICAL VALUES USING MEDIAN

```
# 5.6) Fill missing numerical values using median
for col in numeric_columns:
    median_value = df_clean[col].median()
    if pd.notna(median_value):
        df_clean[col] = df_clean[col].fillna(median_value)
```

5.7) FILL MISSING CATEGORICAL VALUES USING MODE

```
# 5.7) Fill missing categorical values using mode
categorical_columns = df_clean.select_dtypes(
    include=['object', 'category', 'string']
).columns

for col in categorical_columns:
    if df_clean[col].isnull().any():
        mode_value = df_clean[col].mode()
        if not mode_value.empty:
            df_clean[col] = df_clean[col].fillna(mode_value[0])
        else:
            df_clean[col] = df_clean[col].fillna("Unknown")

print("\n========== DATA CLEANING COMPLETED ==========\n")
print(df_clean.isnull().sum())

print("\nCleaned Dataset Shape:")
print(df_clean.shape)
```

6) CAMPAIGN PERFORMANCE ANALYSIS

6.1) SCATTER PLOT - ACQUISITION COST VS ROI

```
# 6.1) SCATTER PLOT - ACQUISITION COST VS ROI
plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=df_clean,
    x='Acquisition_Cost',
    y='ROI',
    hue='Campaign_Type'
)

plt.title('Acquisition Cost vs ROI')
plt.xlabel('Acquisition Cost')
plt.ylabel('Return on Investment (ROI)')
plt.tight_layout()
plt.show()
```

6.2) BAR CHART - AVERAGE CONVERSION RATE BY CHANNEL

```
# 6.2) BAR CHART - AVERAGE CONVERSION RATE BY CHANNEL
plt.figure(figsize=(12, 6))

sns.barplot(
    data=df_clean,
    x=channel_column,
    y='Conversion_Rate',
    hue='Campaign_Type',
    errorbar=None
)

plt.title('Average Conversion Rate by Channel and Campaign Type')
plt.xlabel('Channel Used')
plt.ylabel('Average Conversion Rate')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

6.3) BOX PLOT - ENGAGEMENT SCORE BY CAMPAIGN TYPE

```
# 6.3) BOX PLOT - ENGAGEMENT SCORE BY CAMPAIGN TYPE
plt.figure(figsize=(10, 6))

sns.boxplot(
    data=df_clean,
    x='Campaign_Type',
    y='Engagement_Score'
)
```

```
plt.title('Engagement Score by Campaign Type')
plt.xlabel('Campaign Type')
plt.ylabel('Engagement Score')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

6.4) BAR CHART - AVERAGE ROI BY COMPANY

```
# 6.4) BAR CHART - AVERAGE ROI BY COMPANY
company_roi = df_clean.groupby(
    'Company'
)['ROI'].mean().sort_values(ascending=False)

plt.figure(figsize=(12, 6))

sns.barplot(
    x=company_roi.index,
    y=company_roi.values,
    errorbar=None
)

plt.title('Average ROI by Company')
plt.xlabel('Company')
plt.ylabel('Average ROI')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

6.5) HEATMAP - ENGAGEMENT SCORE AND CONVERSION RATE

```
# 6.5) HEATMAP - ENGAGEMENT SCORE AND CONVERSION RATE
correlation_data = df_clean[
    ['Engagement_Score', 'Conversion_Rate']
]

correlation = correlation_data.corr()

plt.figure(figsize=(7, 5))

sns.heatmap(
    correlation,
    annot=True,
    cmap='coolwarm',
    fmt='.2f'
)

plt.title('Correlation: Engagement Score and Conversion Rate')
plt.tight_layout()
plt.show()
```

7) CUSTOMER SEGMENTATION ANALYSIS

7.1) COUNT PLOT - TARGET AUDIENCE DISTRIBUTION

```
# 7.1) COUNT PLOT - TARGET AUDIENCE DISTRIBUTION
plt.figure(figsize=(12, 6))

sns.countplot(
    data=df_clean,
    x='Target_Audience',
    order=df_clean['Target_Audience'].value_counts().index
)

plt.title('Distribution of Target Audience')
plt.xlabel('Target Audience')
plt.ylabel('Number of Campaigns')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

7.2) BAR CHART - CUSTOMER SEGMENT CONVERSION RATE BY LANGUAGE

```
# 7.2) BAR CHART - CUSTOMER SEGMENT CONVERSION RATE BY LANGUAGE
segment_language = df_clean.groupby(
    ['Language', 'Customer_Segment']
)['Conversion_Rate'].mean().reset_index()

plt.figure(figsize=(14, 7))

sns.barplot(
    data=segment_language,
    x='Language',
    y='Conversion_Rate',
    hue='Customer_Segment',
    errorbar=None
)

plt.title('Average Conversion Rate by Customer Segment and Language')
plt.xlabel('Language')
plt.ylabel('Average Conversion Rate')
plt.xticks(rotation=45)
plt.legend(
    title='Customer Segment',
    bbox_to_anchor=(1.02, 1),
    loc='upper left'
)
plt.tight_layout()
plt.show()
```

7.3) IDENTIFY THE HIGHEST-CONVERTING SEGMENT FOR EACH LANGUAGE

```
# 7.3) Identify the highest-converting segment for each language
best_segments = segment_language.loc[
    segment_language.groupby('Language')['Conversion_Rate'].idxmax()
]

print("\n========== BEST CUSTOMER SEGMENT BY LANGUAGE ==========\n")
print(best_segments)
```

7.4) BOX PLOT - ACQUISITION COST BY CUSTOMER SEGMENT

```
# 7.4) BOX PLOT - ACQUISITION COST BY CUSTOMER SEGMENT
plt.figure(figsize=(14, 7))

sns.boxplot(
    data=df_clean,
    x='Customer_Segment',
    y='Acquisition_Cost',
    hue=channel_column
)

plt.title('Acquisition Cost by Customer Segment and Channel')
plt.xlabel('Customer Segment')
plt.ylabel('Acquisition Cost')
plt.xticks(rotation=45)
plt.legend(
    title='Channel Used',
    bbox_to_anchor=(1.02, 1),
    loc='upper left'
)
plt.tight_layout()
plt.show()
```

7.5) BAR CHART - AVERAGE CONVERSION RATE BY LANGUAGE

```
# 7.5) BAR CHART - AVERAGE CONVERSION RATE BY LANGUAGE
language_conversion = df_clean.groupby(
    'Language'
)['Conversion_Rate'].mean().sort_values(ascending=False)

plt.figure(figsize=(10, 6))

sns.barplot(
    x=language_conversion.index,
    y=language_conversion.values,
    errorbar=None
)
```

```
plt.title('Average Conversion Rate by Language')
plt.xlabel('Language')
plt.ylabel('Average Conversion Rate')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

8) CHANNEL EFFECTIVENESS ANALYSIS

8.1) BAR CHART - ENGAGEMENT SCORE BY CHANNEL

```
# 8.1) BAR CHART - ENGAGEMENT SCORE BY CHANNEL
plt.figure(figsize=(12, 6))

sns.barplot(
    data=df_clean,
    x=channel_column,
    y='Engagement_Score',
    hue='Campaign_Type',
    errorbar=None
)

plt.title('Engagement Score by Channel and Campaign Type')
plt.xlabel('Channel Used')
plt.ylabel('Average Engagement Score')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

8.2) PIE CHART - TOTAL ROI BY CHANNEL

```
# 8.2) PIE CHART - TOTAL ROI BY CHANNEL
channel_roi = df_clean.groupby(
    channel_column
)['ROI'].sum()

plt.figure(figsize=(9, 9))

plt.pie(
    channel_roi,
    labels=channel_roi.index,
    autopct='%1.1f%%',
    startangle=90
)

plt.title('Distribution of Total ROI by Channel')
plt.show()
```

8.3) SCATTER PLOT - CLICKS VS IMPRESSIONS BY CHANNEL

```
# 8.3) SCATTER PLOT - CLICKS VS IMPRESSIONS BY CHANNEL
plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=df_clean,
    x='Impressions',
    y='Clicks',
    hue=channel_column
)

plt.title('Clicks vs Impressions by Channel')
plt.xlabel('Impressions')
plt.ylabel('Clicks')
plt.tight_layout()
plt.show()
```

9) TIME-BASED ANALYSIS

9.1) HISTOGRAM - CAMPAIGN DURATION DISTRIBUTION

```
# 9.1) HISTOGRAM - CAMPAIGN DURATION DISTRIBUTION
plt.figure(figsize=(10, 6))

plt.hist(
    df_clean['Duration'].dropna(),  # Drop any leftover NaNs
    bins=20,
    edgecolor='black'
)

plt.title('Distribution of Campaign Duration')
plt.xlabel('Duration (Days)')
plt.ylabel('Number of Campaigns')
plt.tight_layout()
plt.show()
```

9.2) LINE CHART - CONVERSION RATE OVER DATE BY COMPANY

```
# 9.2) LINE CHART - CONVERSION RATE OVER DATE BY COMPANY
company_date = df_clean.groupby(
    ['Date', 'Company']
)['Conversion_Rate'].mean().reset_index()

plt.figure(figsize=(14, 7))

sns.lineplot(
    data=company_date,
    x='Date',
    y='Conversion_Rate',
    hue='Company',
    marker='o'
)

plt.title('Conversion Rate Over Time by Company')
plt.xlabel('Date')
plt.ylabel('Average Conversion Rate')
plt.xticks(rotation=45)
plt.legend(
    title='Company',
    bbox_to_anchor=(1.02, 1),
    loc='upper left'
)
plt.tight_layout()
plt.show()
```

9.3) LINE CHART - ENGAGEMENT SCORE OVER DATE

```
# 9.3) LINE CHART - ENGAGEMENT SCORE OVER DATE
date_engagement = df_clean.groupby(
    'Date'
)['Engagement_Score'].mean().reset_index()

plt.figure(figsize=(12, 6))

sns.lineplot(
    data=date_engagement,
    x='Date',
    y='Engagement_Score',
    marker='o'
)

plt.title('Engagement Score Trend Over Time')
plt.xlabel('Date')
plt.ylabel('Average Engagement Score')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

10) GEOGRAPHICAL ANALYSIS

10.1) BAR CHART - ACQUISITION COST BY LOCATION

```
# 10.1) BAR CHART - ACQUISITION COST BY LOCATION
location_cost = df_clean.groupby(
    'Location'
)['Acquisition_Cost'].mean().sort_values(ascending=False)

plt.figure(figsize=(12, 6))

sns.barplot(
    x=location_cost.index,
    y=location_cost.values,
    errorbar=None
)

plt.title('Average Acquisition Cost by Location')
plt.xlabel('Location')
plt.ylabel('Average Acquisition Cost')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

print("\nLocation with Highest Average Acquisition Cost:")
print(location_cost.idxmax())
```

10.2) BAR CHART - CONVERSION RATE BY LOCATION

```
# 10.2) BAR CHART - CONVERSION RATE BY LOCATION
location_conversion = df_clean.groupby(
    ['Location', 'Target_Audience']
)['Conversion_Rate'].mean().reset_index()

plt.figure(figsize=(14, 7))

sns.barplot(
    data=location_conversion,
    x='Location',
    y='Conversion_Rate',
    hue='Target_Audience',
    errorbar=None
)

plt.title('Conversion Rate by Location and Target Audience')
plt.xlabel('Location')
plt.ylabel('Average Conversion Rate')
plt.xticks(rotation=45)
plt.legend(
    title='Target Audience',
```

```
    bbox_to_anchor=(1.02, 1),
    loc='upper left'
)
plt.tight_layout()
plt.show()
```

10.3) PIE CHART - PROPORTION OF ROI BY LOCATION

```
# 10.3) PIE CHART - PROPORTION OF ROI BY LOCATION
location_roi = df_clean.groupby(
    'Location'
)['ROI'].sum()

plt.figure(figsize=(9, 9))

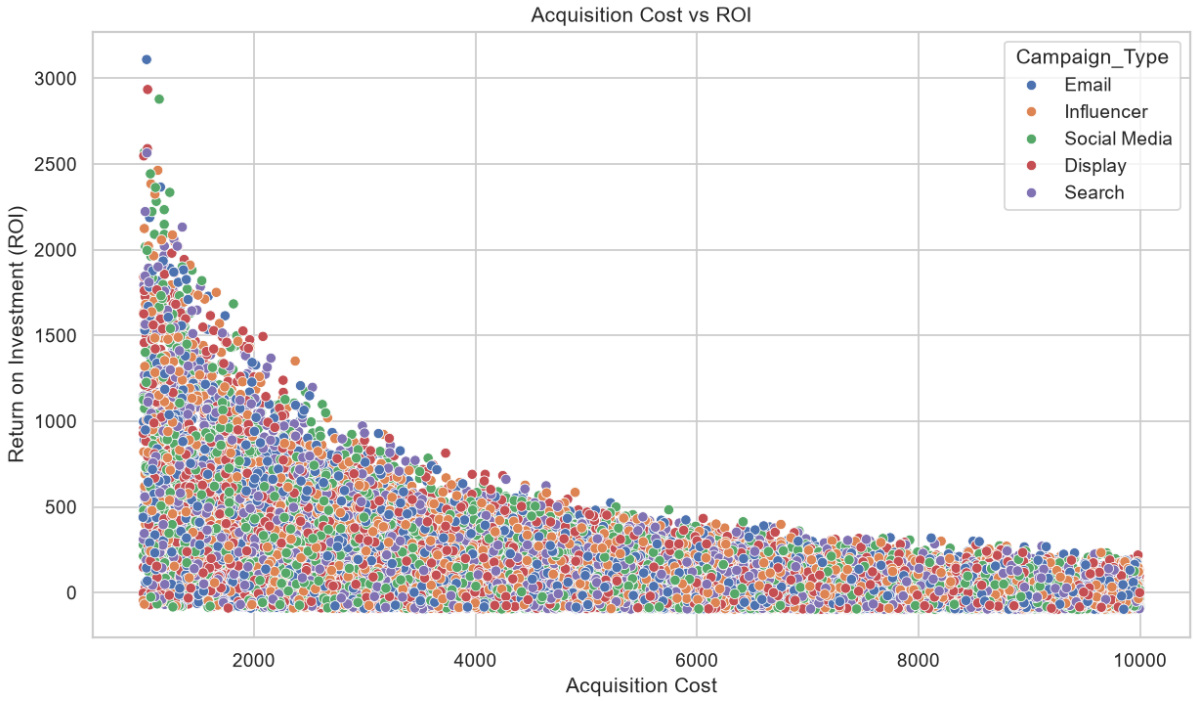
plt.pie(
    location_roi,
    labels=location_roi.index,
    autopct='%1.1f%%',
    startangle=90
)

plt.title('Proportion of Total ROI by Location')
plt.show()
```

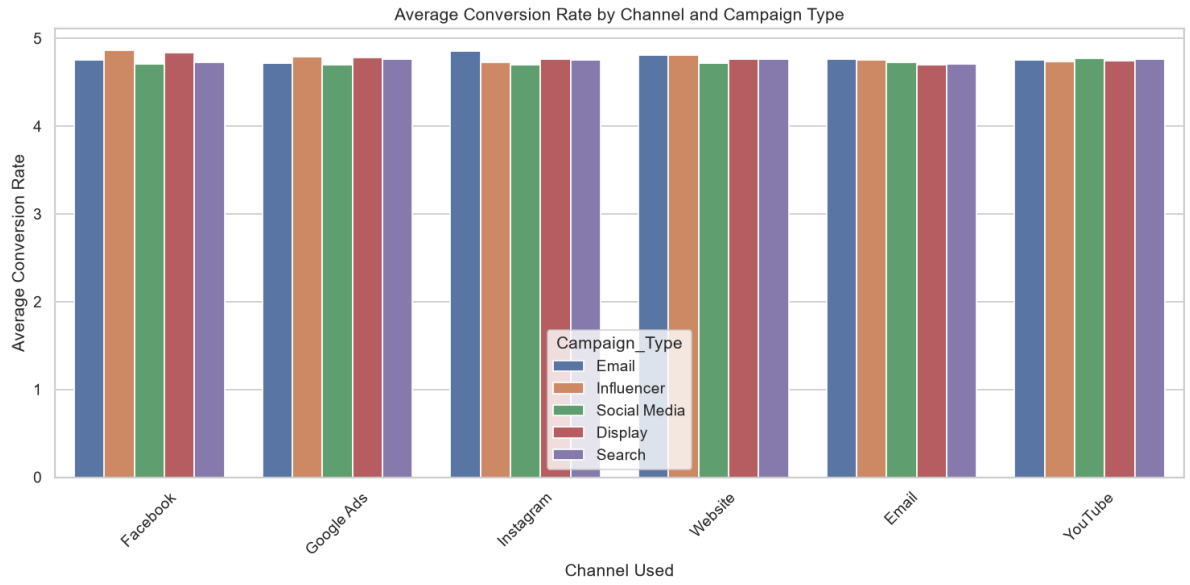
11) KEY PERFORMANCE FINDINGS

```
print("\n============================================")
print("       MARKETING CAMPAIGN ANALYSIS REPORT")
print("============================================")
print("\nTotal Number of Campaigns:")
print(len(df_clean))
print("\nNumber of Unique Campaigns:")
print(df_clean['Campaign_ID'].nunique())
print("\nAverage Conversion Rate:")
print(round(df_clean['Conversion_Rate'].mean(), 2))
print("\nAverage Acquisition Cost:")
print(round(df_clean['Acquisition_Cost'].mean(), 2))
print("\nAverage ROI:")
print(round(df_clean['ROI'].mean(), 2))
print("\nAverage Engagement Score:")
print(round(df_clean['Engagement_Score'].mean(), 2))
print("\nCampaign Type with Highest Average ROI:")
print(
    df_clean.groupby('Campaign_Type')['ROI']
    .mean().idxmax()
)
print("\nCompany with Highest Average ROI:")
print(
    df_clean.groupby('Company')['ROI']
    .mean().idxmax()
)
print("\nChannel with Highest Average Conversion Rate:")
print(
    df_clean.groupby(channel_column)['Conversion_Rate']
    .mean().idxmax()
)
print("\nLocation with Highest Average Acquisition Cost:")
print(
    df_clean.groupby('Location')['Acquisition_Cost']
    .mean().idxmax()
)
print("\nCustomer Segment with Highest Average Conversion Rate:")
print(
    df_clean.groupby('Customer_Segment')['Conversion_Rate']
    .mean().idxmax()
)
```

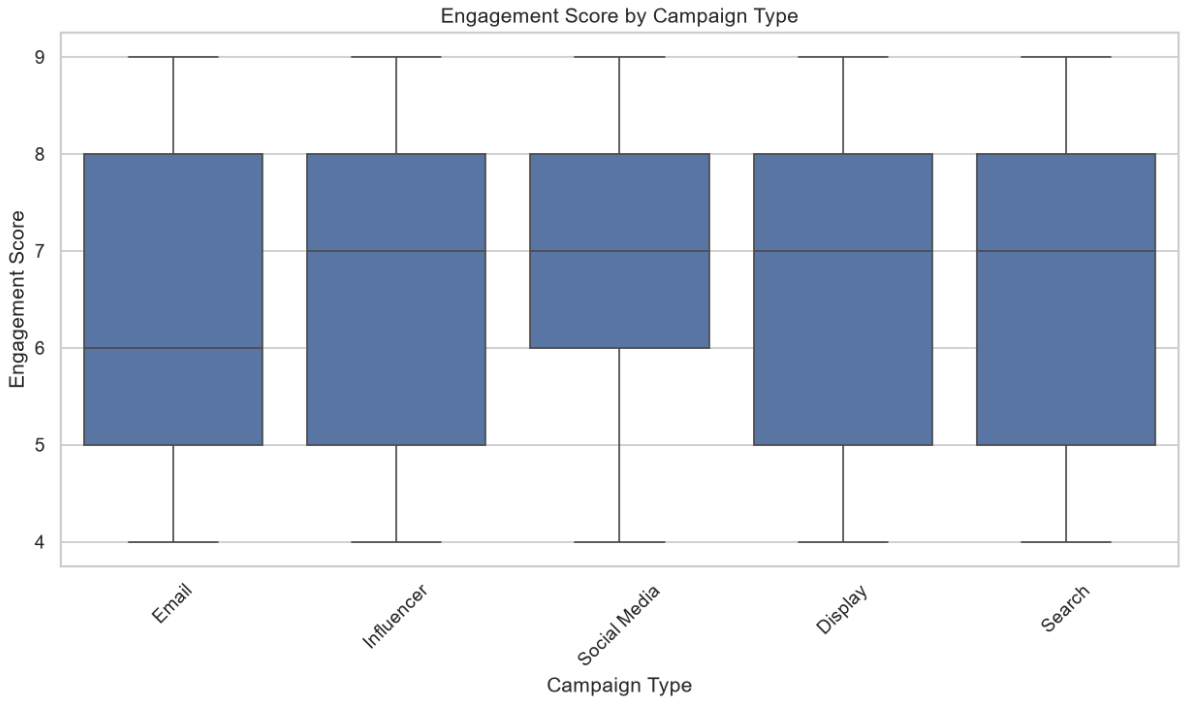
OUTPUT 6.1) SCATTER PLOT - ACQUISITION COST VS ROI



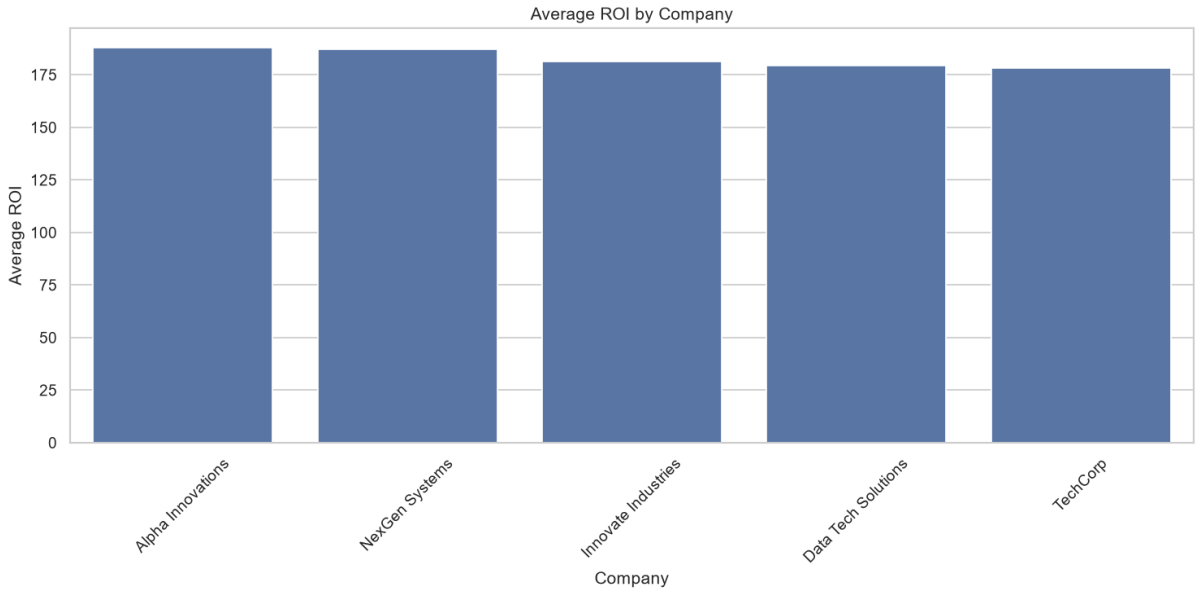
OUTPUT 6.2) BAR CHART - AVERAGE CONVERSION RATE BY CHANNEL



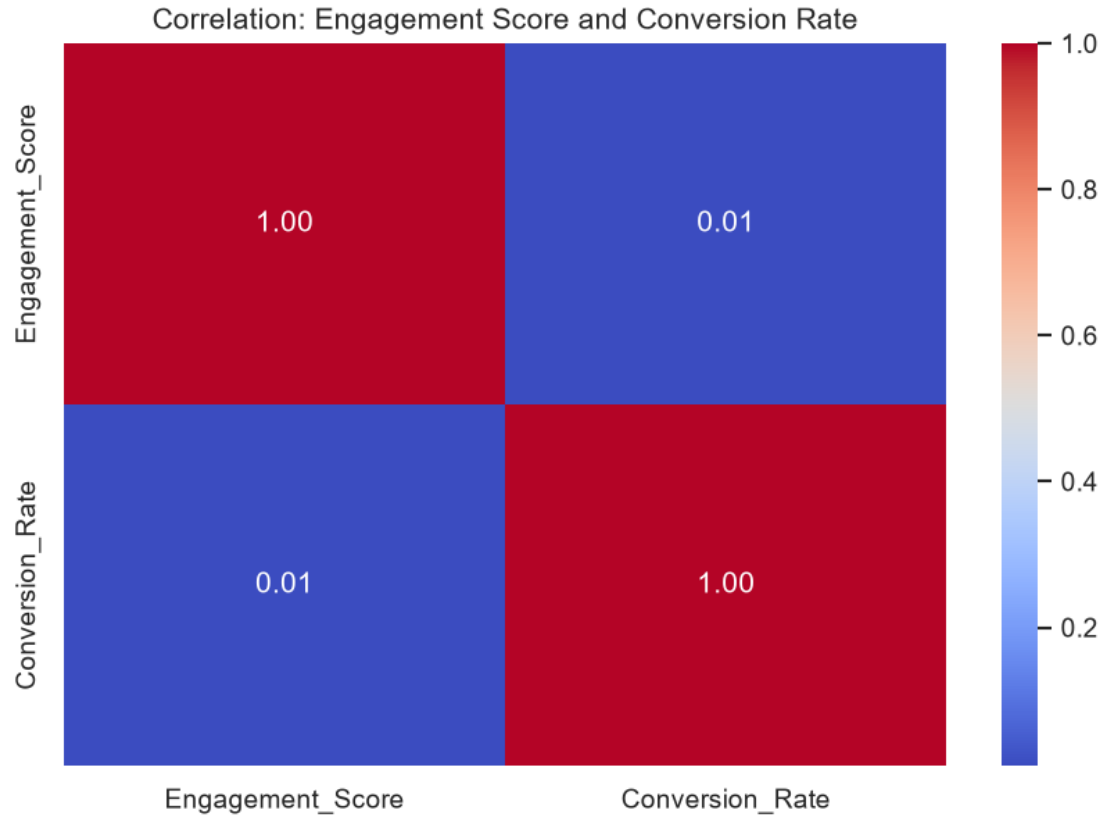
OUTPUT 6.3) BOX PLOT - ENGAGEMENT SCORE BY CAMPAIGN TYPE



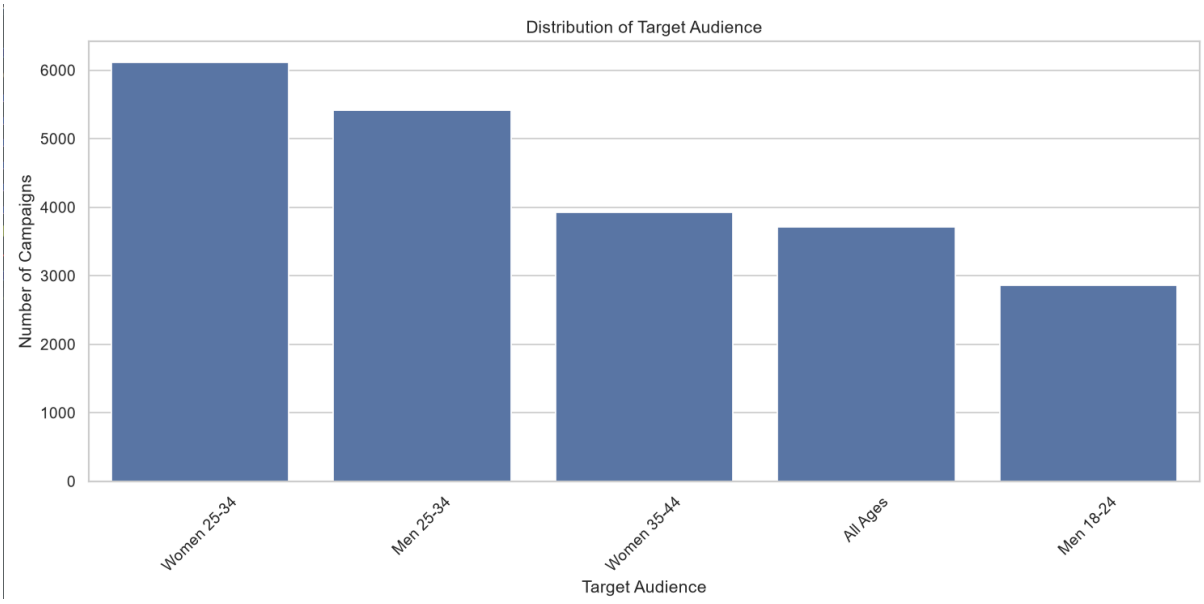
OUTPUT 6.4) BAR CHART - AVERAGE ROI BY COMPANY



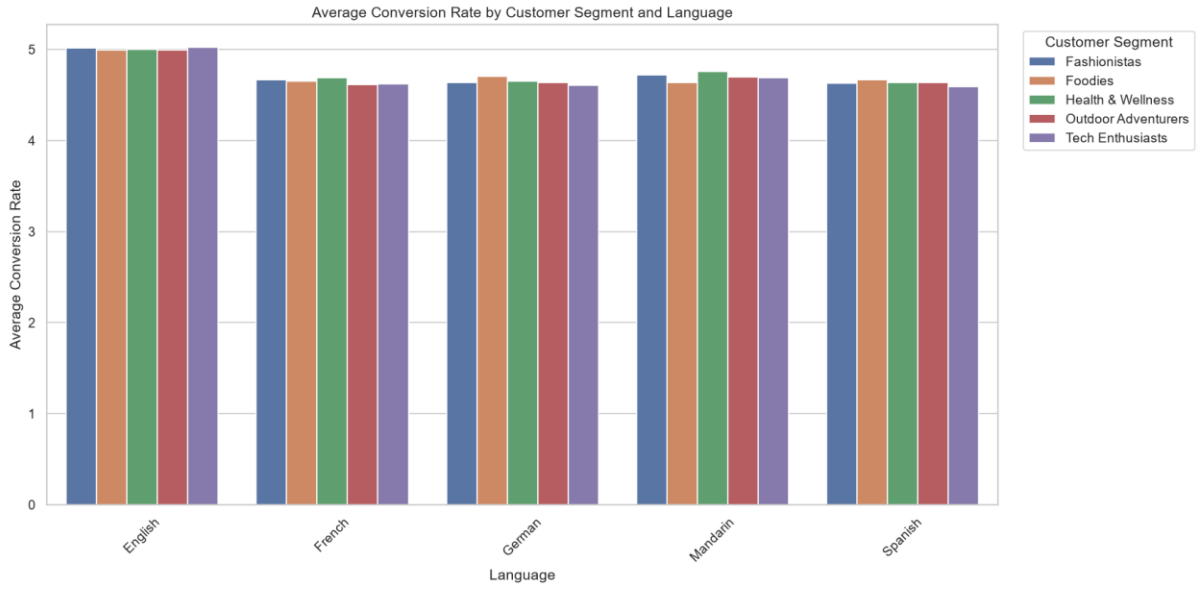
OUTPUT 6.5) HEATMAP - ENGAGEMENT SCORE AND CONVERSION RATE



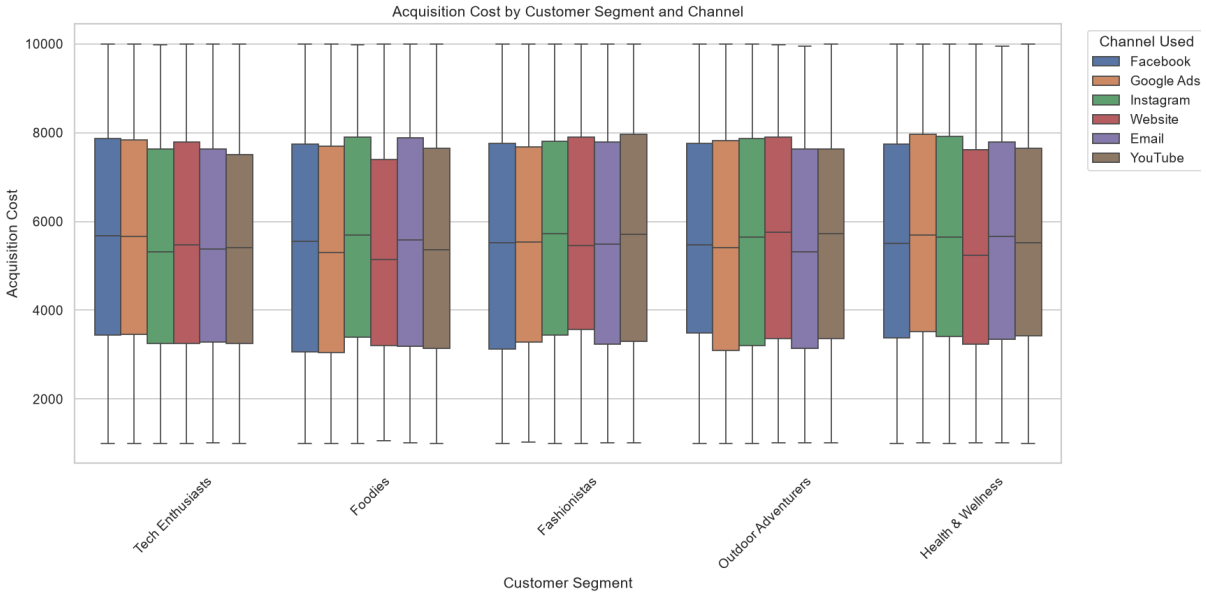
OUTPUT 7.1) COUNT PLOT - TARGET AUDIENCE DISTRIBUTION



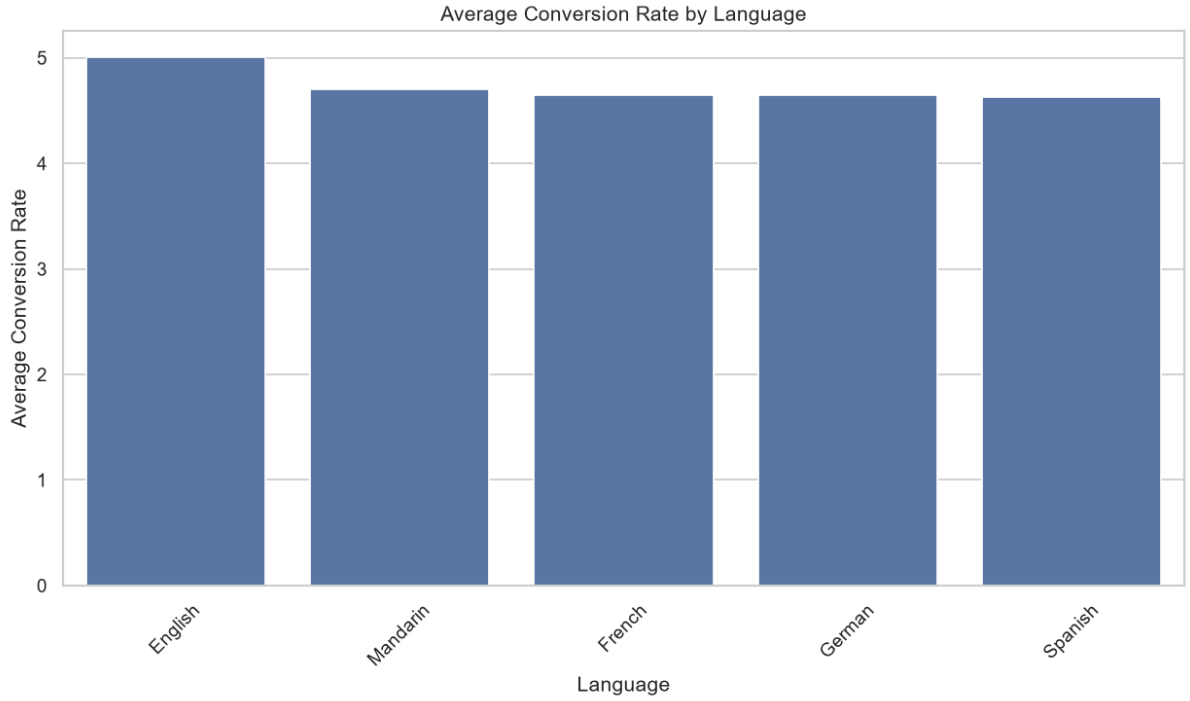
OUTPUT 7.2) BAR CHART - CUSTOMER SEGMENT CONVERSION RATE BY LANGUAGE



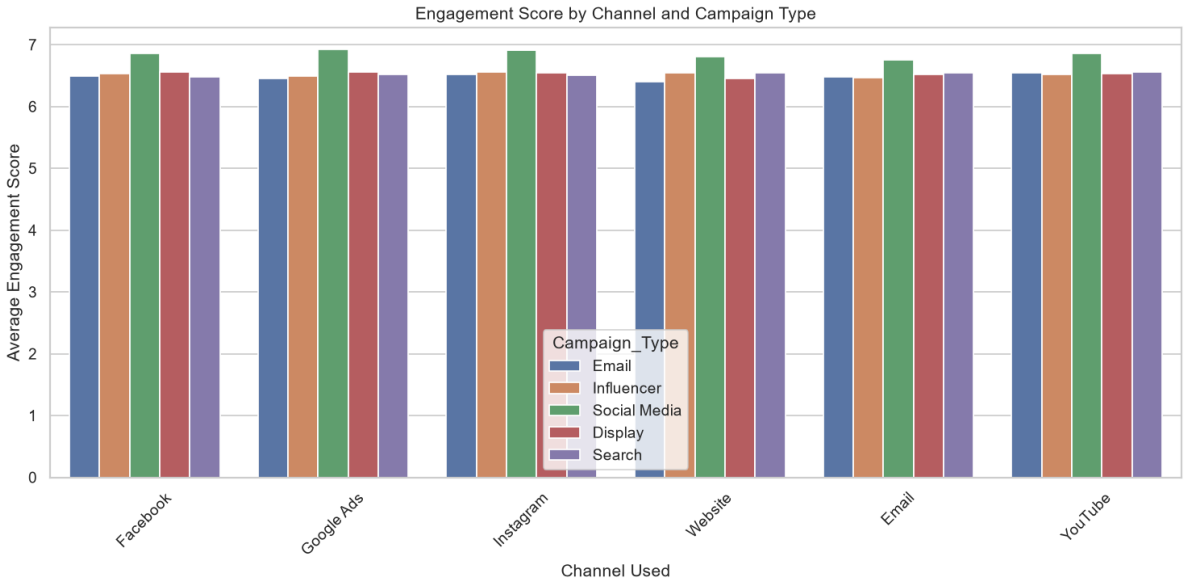
OUTPUT 7.4) BOX PLOT - ACQUISITION COST BY CUSTOMER SEGMENT



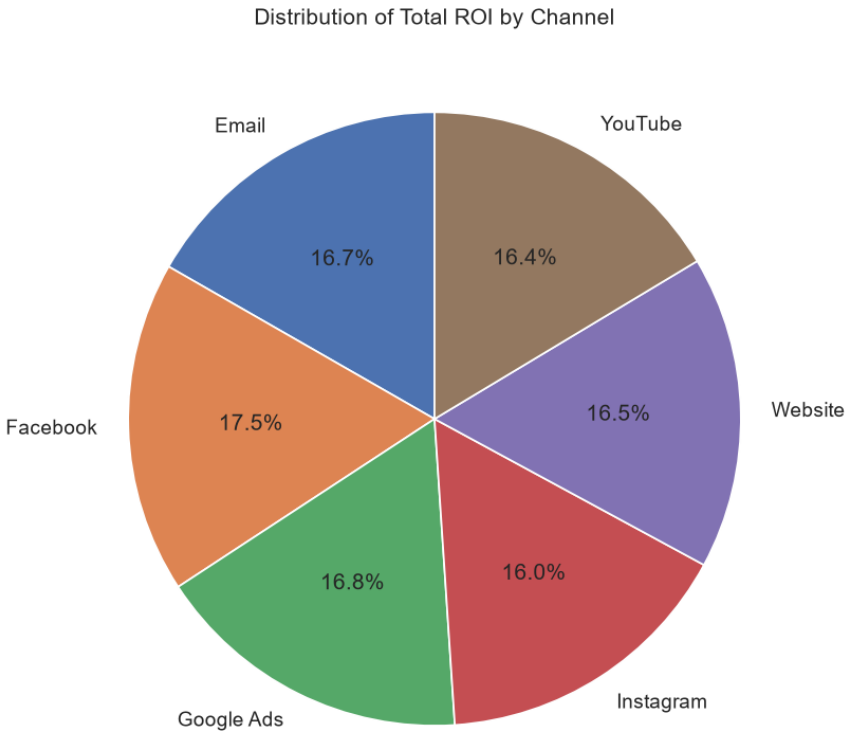
OUTPUT 7.5) BAR CHART - AVERAGE CONVERSION RATE BY LANGUAGE



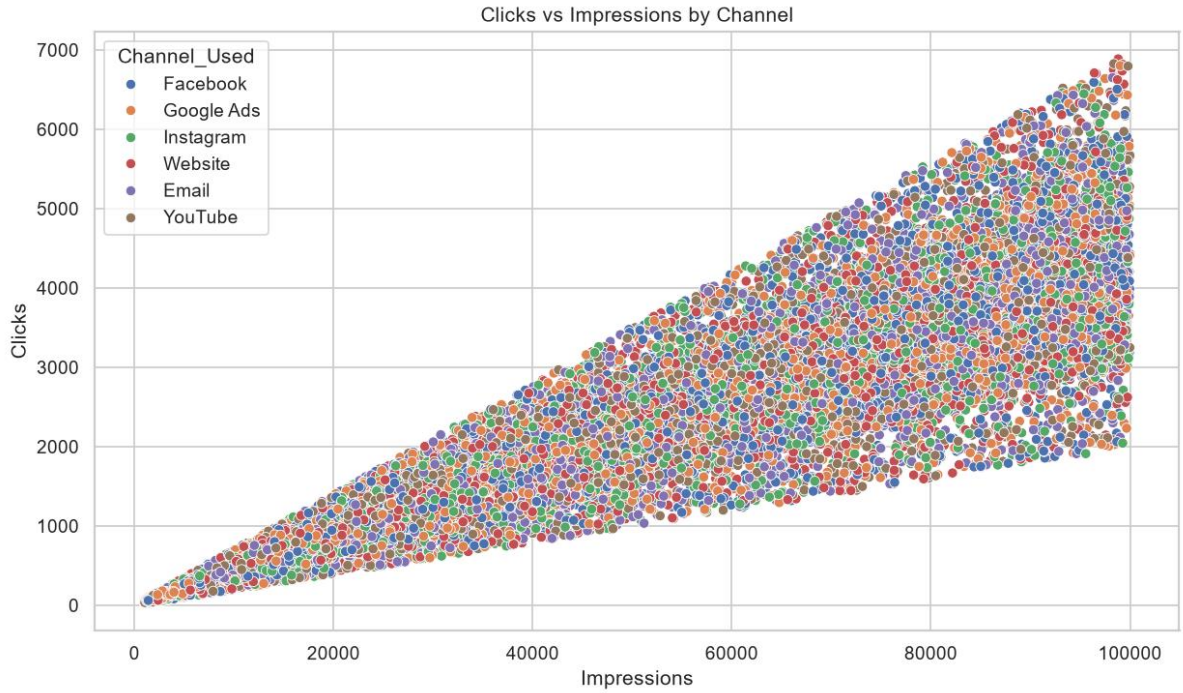
OUTPUT 8.1) BAR CHART - ENGAGEMENT SCORE BY CHANNEL



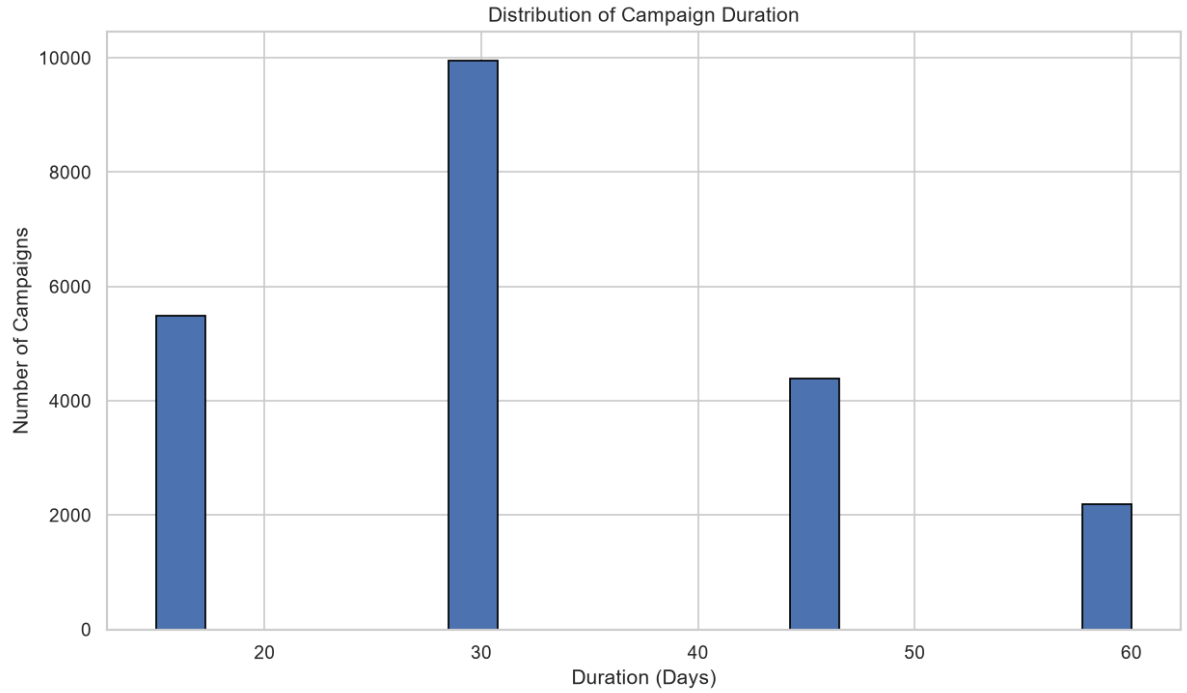
OUTPUT 8.2) PIE CHART - TOTAL ROI BY CHANNEL



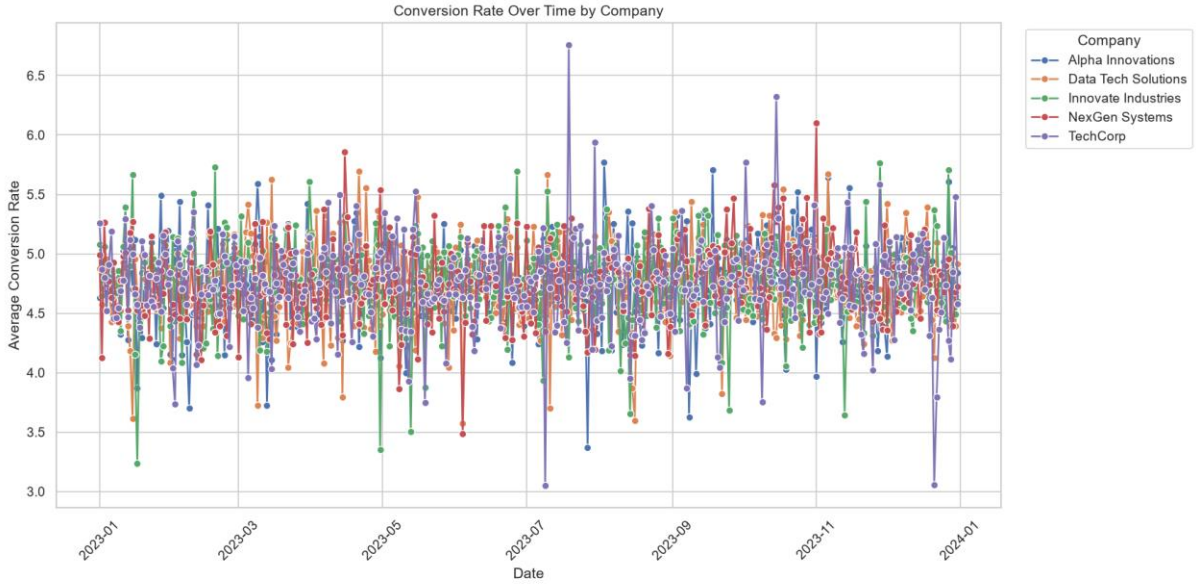
OUTPUT 8.3) SCATTER PLOT - CLICKS VS IMPRESSIONS BY CHANNEL



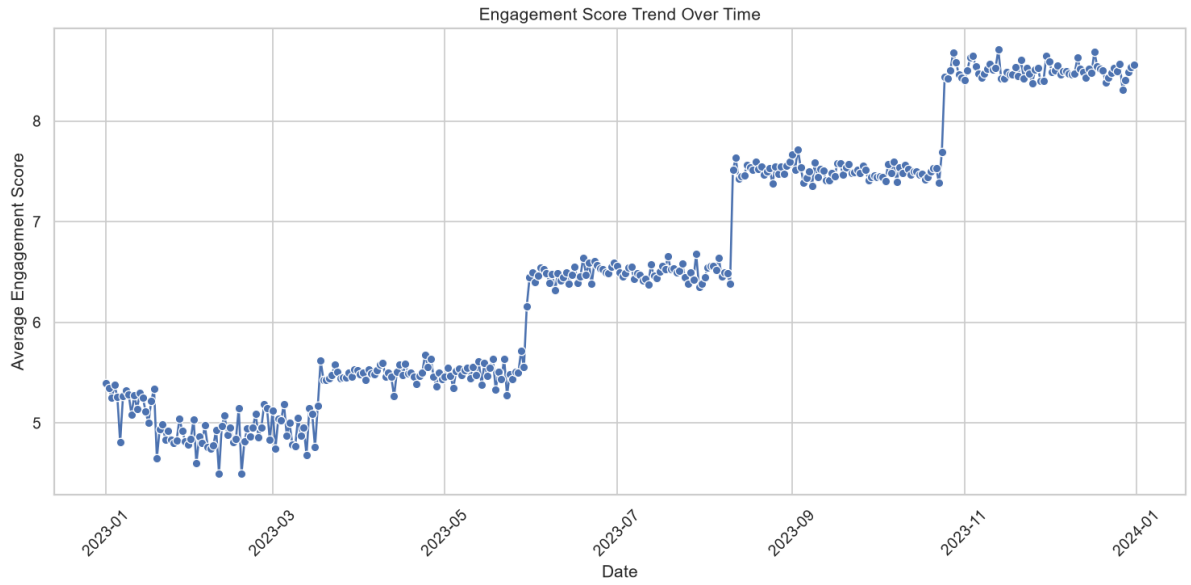
OUTPUT 9.1) HISTOGRAM - CAMPAIGN DURATION DISTRIBUTION



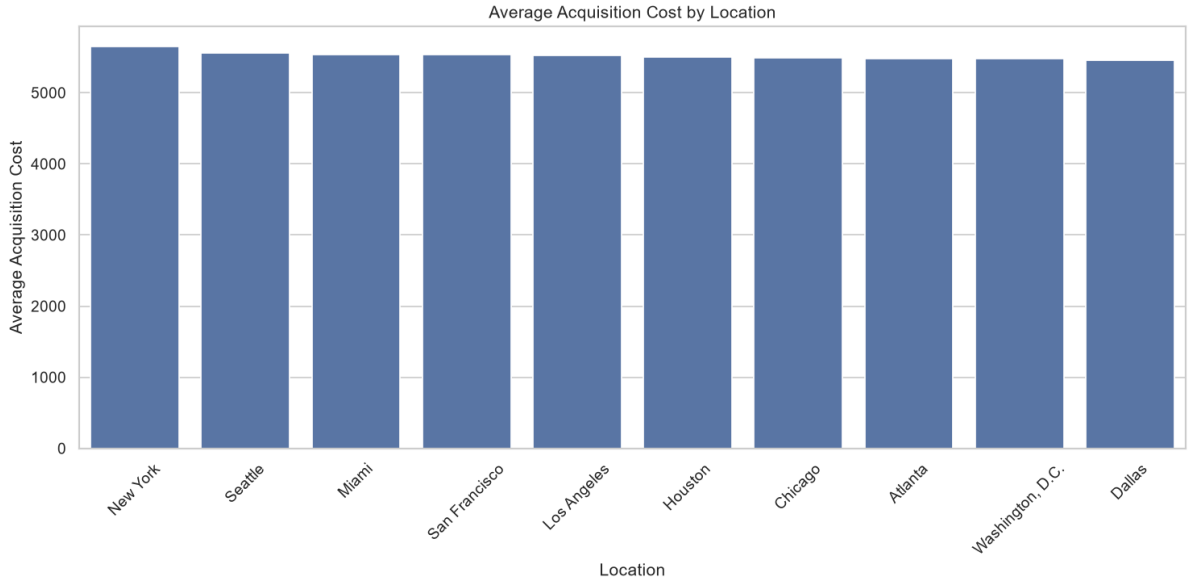
OUTPUT 9.2) LINE CHART - CONVERSION RATE OVER DATE BY COMPANY



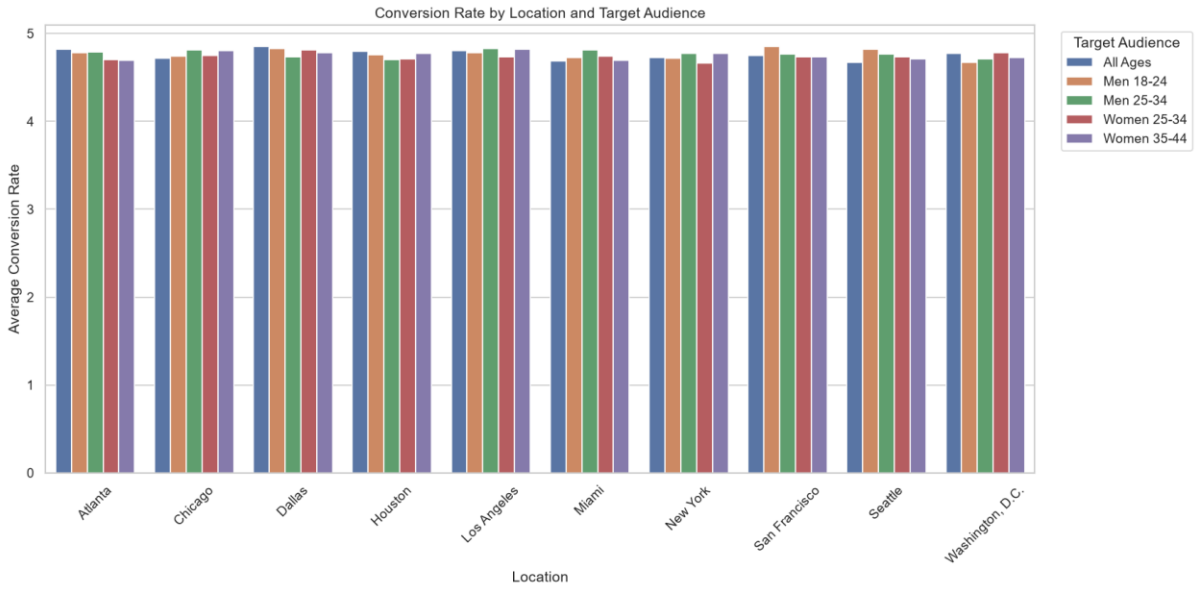
OUTPUT 9.3) LINE CHART - ENGAGEMENT SCORE OVER DATE



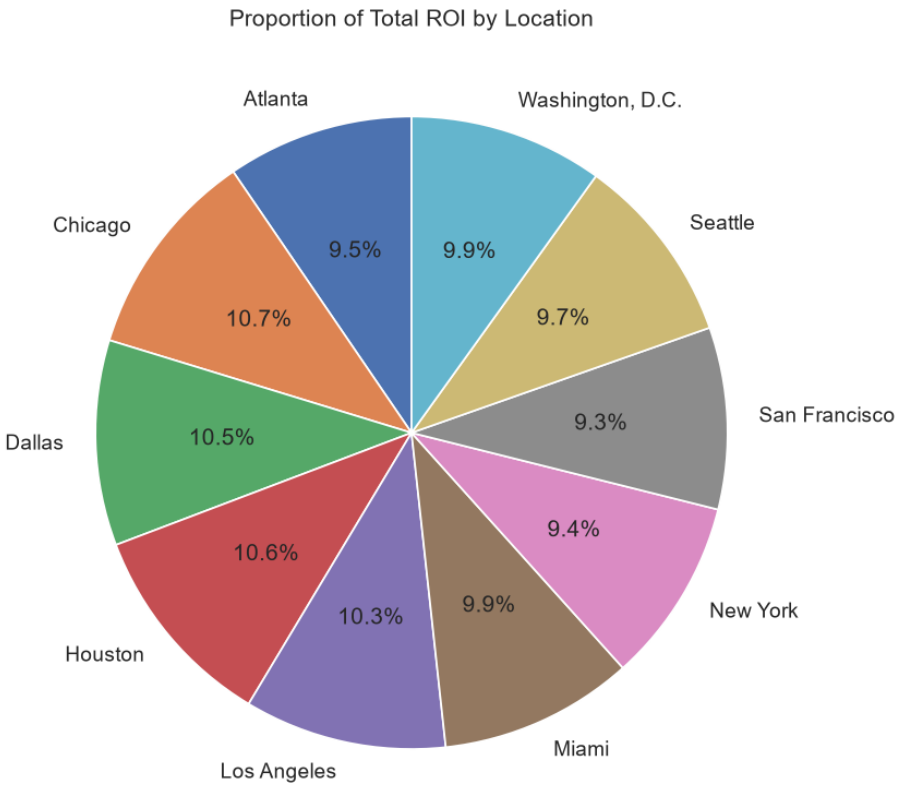
OUTPUT 10.1) BAR CHART - ACQUISITION COST BY LOCATION



OUTPUT 10.2) BAR CHART - CONVERSION RATE BY LOCATION



OUTPUT 10.3) PIE CHART - PROPORTION OF ROI BY LOCATION



PROGRAM OUTPUT (TERMINAL)

```
========== DATASET LOADED SUCCESSFULLY ==========

   Campaign_ID              Company Campaign_Type Target_Audience  ...
Impressions  Engagement_Score  Customer_Segment        Date
0            1             TechCorp         Email     Women 25-34  ...
67836                 5  Tech Enthusiasts  01-01-2023
1            2  Innovate Industries    Influencer     Women 35-44  ...
66361                 4           Foodies  01-01-2023
2            3       NexGen Systems  Social Media     Women 25-34  ...
86240                 8      Fashionistas  01-01-2023
3            4  Innovate Industries         Email     Women 25-34  ...
58251                 6           Foodies  01-01-2023
4            5  Data Tech Solutions    Influencer       Men 25-34  ...
34407                 5  Tech Enthusiasts  01-01-2023

[5 rows x 16 columns]

========== FIRST 5 ROWS ==========

   Campaign_ID              Company Campaign_Type Target_Audience  ...
Impressions  Engagement_Score  Customer_Segment        Date
0            1             TechCorp         Email     Women 25-34  ...
67836                 5  Tech Enthusiasts  01-01-2023
1            2  Innovate Industries    Influencer     Women 35-44  ...
66361                 4           Foodies  01-01-2023
2            3       NexGen Systems  Social Media     Women 25-34  ...
86240                 8      Fashionistas  01-01-2023
3            4  Innovate Industries         Email     Women 25-34  ...
58251                 6           Foodies  01-01-2023
4            5  Data Tech Solutions    Influencer       Men 25-34  ...
34407                 5  Tech Enthusiasts  01-01-2023

[5 rows x 16 columns]

========== NUMBER OF ROWS AND COLUMNS ==========

(22029, 16)

========== COLUMN NAMES ==========

Index(['Campaign_ID', 'Company', 'Campaign_Type', 'Target_Audience',
       'Duration', 'Channel_Used', 'Conversion_Rate', 'Acquisition_Cost',
       'ROI', 'Location', 'Language', 'Clicks', 'Impressions',
       'Engagement_Score', 'Customer_Segment', 'Date'],
      dtype='str')

========== DATA TYPES ==========
```

```
Campaign_ID           int64
Company                 str
Campaign_Type           str
Target_Audience         str
Duration                str
Channel_Used            str
Conversion_Rate     float64
Acquisition_Cost      int64
ROI                 float64
Location                str
Language                str
Clicks                int64
Impressions           int64
Engagement_Score      int64
Customer_Segment        str
Date                    str
dtype: object

========== DATASET INFORMATION ==========

<class 'pandas.DataFrame'>
RangeIndex: 22029 entries, 0 to 22028
Data columns (total 16 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Campaign_ID       22029 non-null  int64
 1   Company           22029 non-null  str
 2   Campaign_Type     22029 non-null  str
 3   Target_Audience   22029 non-null  str
 4   Duration          22029 non-null  str
 5   Channel_Used      22029 non-null  str
 6   Conversion_Rate   22029 non-null  float64
 7   Acquisition_Cost  22029 non-null  int64
 8   ROI               22029 non-null  float64
 9   Location          22029 non-null  str
 10  Language          22029 non-null  str
 11  Clicks            22029 non-null  int64
 12  Impressions       22029 non-null  int64
 13  Engagement_Score  22029 non-null  int64
 14  Customer_Segment  22029 non-null  str
 15  Date              22029 non-null  str
dtypes: float64(2), int64(5), str(9)
memory usage: 2.7 MB

========== DESCRIPTIVE STATISTICS ==========
```

```
        Campaign_ID  Conversion_Rate  Acquisition_Cost           ROI
Clicks   Impressions  Engagement_Score
count  22029.000000     22029.000000      22029.000000  22029.000000
22029.000000  22029.000000      22029.000000
mean   11015.000000         4.757232       5522.740842    182.863648
2223.807572  50610.402787          6.582323
std     6359.368876         0.960393       2597.666260    301.619721
1394.166380  28542.979123          1.458804
min        1.000000         2.015723       1000.000000    -98.300000
30.000000   1001.000000          4.000000
25%     5508.000000         4.130705       3286.000000     -4.080000
1067.000000  25804.000000          5.000000
50%    11015.000000         4.761527       5525.000000     93.650000
2088.000000  50858.000000          7.000000
75%    16522.000000         5.429335       7766.000000    247.310000
3212.000000  75165.000000          8.000000
max    22029.000000         7.469907       9999.000000   3109.790000
6887.000000  99999.000000          9.000000

========== UNIQUE CAMPAIGN IDs ==========

22029

========== UNIQUE LOCATIONS ==========

<StringArray>
[         'Houston', 'Washington, D.C.',            'Miami',
          'Seattle',          'Chicago',      'Los Angeles',
          'Atlanta',           'Dallas',         'New York',
    'San Francisco']
Length: 10, dtype: str

========== UNIQUE CUSTOMER SEGMENTS ==========

<StringArray>
[   'Tech Enthusiasts',             'Foodies',        'Fashionistas',
 'Outdoor Adventurers',   'Health & Wellness']
Length: 5, dtype: str

========== CAMPAIGN TYPE COUNTS ==========

Campaign_Type
Display         4450
Search          4441
Social Media    4412
Email           4388
Influencer      4338
Name: count, dtype: int64
```

```
========== CHANNEL COUNTS ==========

Channel_Used
Facebook      3742
Google Ads    3694
Website       3688
Instagram     3649
YouTube       3632
Email         3624
Name: count, dtype: int64

========== DATA CLEANING COMPLETED ==========

Campaign_ID         0
Company             0
Campaign_Type       0
Target_Audience     0
Duration            0
Channel_Used        0
Conversion_Rate     0
Acquisition_Cost    0
ROI                 0
Location            0
Language            0
Clicks              0
Impressions         0
Engagement_Score    0
Customer_Segment    0
Date                0
dtype: int64

Cleaned Dataset Shape:
(22029, 16)

========== BEST CUSTOMER SEGMENT BY LANGUAGE ==========

    Language   Customer_Segment  Conversion_Rate
4    English   Tech Enthusiasts         5.022731
7     French  Health & Wellness         4.689704
11    German            Foodies         4.705430
17  Mandarin  Health & Wellness         4.756125
21   Spanish            Foodies         4.663610

Location with Highest Average Acquisition Cost:
New York

==========================================
```

```
       MARKETING CAMPAIGN ANALYSIS REPORT
==========================================

Total Number of Campaigns:
22029

Number of Unique Campaigns:
22029

Average Conversion Rate:
4.76

Average Acquisition Cost:
5522.74

Average ROI:
182.86

Average Engagement Score:
6.58

Campaign Type with Highest Average ROI:
Social Media

Company with Highest Average ROI:
Alpha Innovations

Channel with Highest Average Conversion Rate:
Facebook

Location with Highest Average Acquisition Cost:
New York

Customer Segment with Highest Average Conversion Rate:
Health & Wellness
```